

The new_topic Template

Now we'll make a new template called *new_topic.html* to display the form we just created:

```
new_topic.html {% extends "learning_logs/base.html" %}

{% block content %}
    <p>Add a new topic:</p>

    ❶ <form action="{% url 'learning_logs:new_topic' %}" method='post'>
    ❷   {% csrf_token %}
    ❸   {{ form.as_div }}
    ❹   <button name="submit">Add topic</button>
    </form>

{% endblock content %}
```

This template extends *base.html*, so it has the same base structure as the rest of the pages in Learning Log. We use the `<form></form>` tags to define an HTML form ❶. The action argument tells the browser where to send the data submitted in the form; in this case, we send it back to the view function `new_topic()`. The method argument tells the browser to submit the data as a POST request.

Django uses the template tag `{% csrf_token %}` ❷ to prevent attackers from using the form to gain unauthorized access to the server. (This kind of attack is called a *cross-site request forgery*.) Next, we display the form; here you can see how simple Django can make certain tasks, such as displaying a form. We only need to include the template variable `{{ form.as_div }}` for Django to create all the fields necessary to display the form automatically ❸. The `as_div` modifier tells Django to render all the form elements as HTML `<div></div>` elements; this is a simple way to display the form neatly.

Django doesn't create a submit button for forms, so we define one before closing the form ❹.

Linking to the new_topic Page

Next, we include a link to the `new_topic` page on the topics page:

```
topics.html {% extends "learning_logs/base.html" %}

{% block content %}

    <p>Topics</p>

    <ul>
        --snip--
    </ul>

    <a href="{% url 'learning_logs:new_topic' %}">Add a new topic</a>

{% endblock content %}
```
